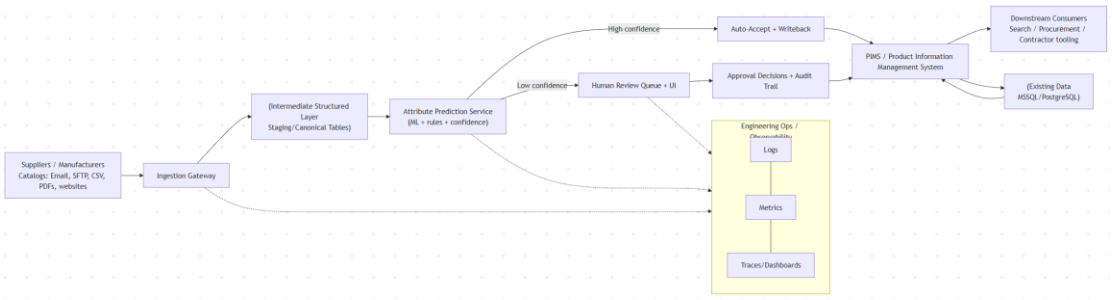
1. Purpose of architectural requirements

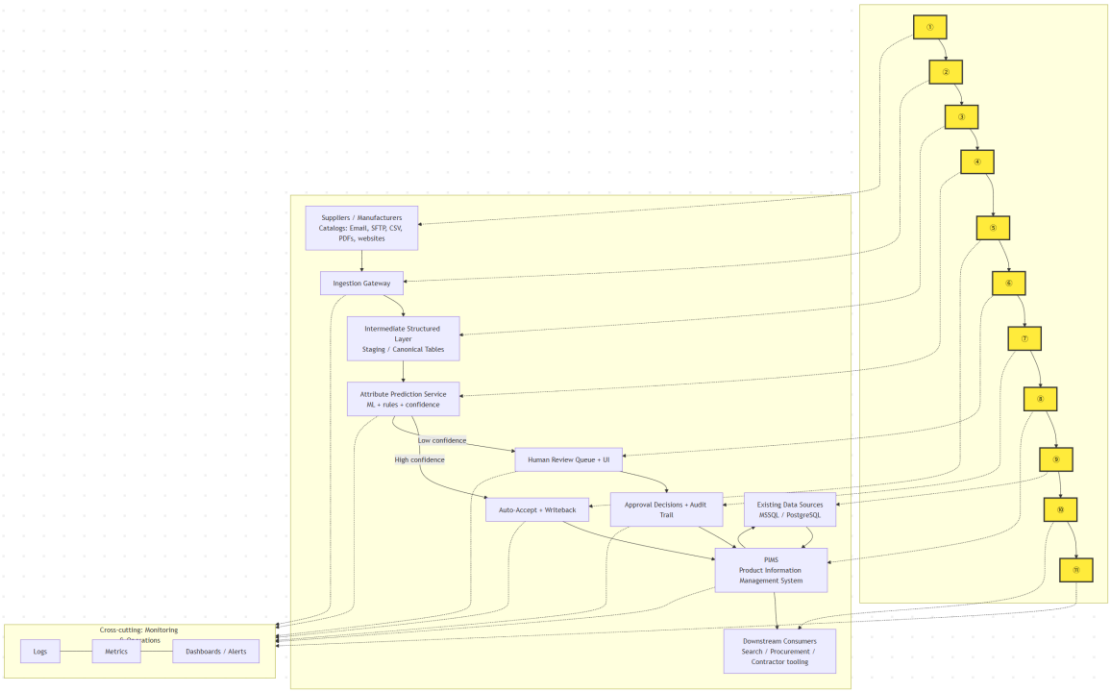
These requirements explain what each architecture part must do and what limits it must follow. They help the client see how data moves end-to-end, and they help the team build, test, and run the system with fewer surprises.

2. eParts Reference Architecture



Link: https://mermaid.live

3. Detailed Requirements by Component



4. System-wide constraints

These apply to every component unless noted.

- Use Auth0 for user sign-in and role-based access control.
- Keep the design cost-effective; avoid costs that grow linearly per tenant when possible
- Start by integrating the existing SQL Server data source and plan for Azure SQL or Managed Instance later
- Deliver breadth-first first: show a small but full end-to-end flow before going deep in one area
- Do a structured evaluation before locking platform tech choices (portability matters)
- Support privacy rules, including deleting customer or tenant data on request

5. System-wide quality attributes

We focus on security, extensibility, usability, interoperability, portability

- Security: only authorized users can see or change data; sensitive actions are logged
- Extensibility: adding a new supplier format or a new attribute should not require rewriting the whole pipeline
- Usability: ops users can review and approve items quickly, without needing technical skills
- Interoperability: integrate with existing eParts systems and data sources
- Portability: core parts can be migrated if needed, to reduce vendor lock-in risk

6. Detailed requirements by architecture component

6.1 Suppliers / Manufacturers

Constraints

- Treat suppliers as external parties; never assume their files follow our schema

Non-functional

- Handle messy inputs safely; do not break the pipeline because one supplier sends a bad file

Requirements

- Identify every submission using a supplier ID or a clear supplier name
- Support common intake channels: Email, SFTP, CSV, PDFs, websites
- Keep the original file as “raw evidence” for later audit and reprocessing

6.2 Ingestion Gateway

Constraints

- Follow the breadth-first approach: deliver one working path end-to-end early

Non-functional

- Reliable retries for temporary failures
- Clear error messages for ops users

Requirements

- Accept files from SFTP and storage-style drops, in formats like CSV and text
- Validate basic file rules before processing: file type, size, virus scan, required metadata
- Create an ingestion record for every submission: supplier, time, source channel, file name, status
- If parsing fails, mark the submission as failed with a reason and keep the raw file for debugging
- Log file receipts and key ingestion steps

6.3 Intermediate Structured Layer (Staging / Canonical Tables)

Constraints

- Support deletion requests when needed

Non-functional

- Data should be easy to query and easy to re-run through the pipeline
- Keep version history so we can compare “before vs after”

Requirements

- Convert raw inputs into a standard table format with consistent column names
- Store parsing outputs separately from predictions, so we can re-run ML without re-parsing
- Track lineage: raw submission → extracted fields → normalized records → predicted attributes → final writeback
- Support schema-based transformation using defined mappings
- Support data deletion by supplier submission or tenant/customer scope

6.4 Attribute Prediction Service (ML + rules + confidence)

Constraints

- ML workflow should support a registry and controlled promotion
- Retraining must be manually triggered; fully automated retraining is not allowed

Non-functional

- Predictions must be reproducible: same model version + same input gives the same output
- Track model quality drift over time

Requirements

- Produce predicted attributes plus a confidence score for each attribute
- Combine simple rules with ML when rules are safer or clearer
- Record which model version produced each prediction
- Monitor model performance and drift with live telemetry
- Provide a manual process to retrain and promote a new model version

6.5 Confidence Routing (High vs Low confidence)

Constraints

- Keep decisions explainable to ops users; do not hide why something is low confidence

Non-functional

- Stable behavior across releases; avoid frequent threshold “surprises”

Requirements

- Define a clear confidence threshold for auto-accept vs human review
- Route high-confidence items to auto-accept
- Route low-confidence items to the review queue
- Allow ops leads to adjust thresholds with approval, and log the change

6.6 Auto-Accept + Writeback

Constraints

- Respect access control and governance rules

Non-functional

- Idempotent writeback: retrying the same writeback should not create duplicates

Requirements

- Only auto-write attributes that pass the confidence threshold
- Validate required fields before writeback
- Record what fields were written, when, and by which model version
- If writeback fails, retry safely and alert ops

6.7 Human Review Queue + UI

Constraints

- Users must authenticate via Auth0

Non-functional

- Easy to use for non-technical users
- Fast enough so the queue does not become a bottleneck

Requirements

- Show one “work item” per low-confidence case, with the source evidence (text/PDF snippet if available)

- Let reviewers accept, edit, or reject predicted attributes
- Provide search and filters by supplier, date, status, category
- Support basic assignment: unassigned, assigned to me, completed
- Log every review action as part of the audit trail

6.8 Approval Decisions + Audit Trail

Constraints

- Track user activity for audit purposes

Non-functional

- Audit records must be tamper-resistant and queryable

Requirements

- Record who changed what, when, and the before/after values
- Record the reason when provided and link it to the supplier submission
- Support reporting: review time, correction rate, top failing suppliers
- Keep audit logs even if the final writeback fails, so we can debug

6.9 PIMS Integration

PIMS means Product Information Management System. It is the system where the final product catalog data lives and where downstream tools read from.

Constraints

- Interoperate with existing eParts systems

Non-functional

- Safe integration: avoid partial updates that leave catalog data inconsistent

Requirements

- Write approved attributes into PIMS (from auto-accept or human approval)
- Support reading reference data from PIMS to help normalization and mapping
- Handle conflicts: if PIMS data changed since ingestion, detect it and require

review

- Provide clear error responses when PIMS is unavailable

6.10 Existing Data Sources (MSSQL / PostgreSQL)

Constraints

- Initial primary data source includes existing SQL Server, with a migration path

Non-functional

- Keep read load controlled so we do not hurt production systems

Requirements

- Read needed reference data for normalization and validation
- Cache common reference data to reduce repeated DB calls
- Track the source and timestamp of reference data used during processing

6.11 Downstream Consumers

Constraints

- Provide clean interfaces so multiple tools can consume the same catalog data

Non-functional

- Data must be consistent and timely enough for business use

Requirements

- Downstream tools read from PIMS as the source of truth
- Provide a way to measure freshness: last update time per catalog or supplier
- Provide a simple “release note” view: what changed this week in catalog attributes

6.12 Engineering Ops / Observability

Constraints

- Audit and governance require logging user actions and key system events

Non-functional

- Quick troubleshooting: engineers can find failures without digging through raw servers

Requirements

- Logs: capture ingestion status, parsing errors, prediction failures, writeback errors
- Metrics: track volume, success rate, queue length, auto-accept rate, review time
- Dashboards: show end-to-end health and "where items get stuck"
- Alerts: notify on spikes in failures or backlog growth
- Keep enough history to compare model versions and supplier trends

7. Example Walkthrough: One Supplier Submission

Use this short walkthrough to show how the system works end to end.

- A supplier sends a catalog file through Email, SFTP, or upload.
- The system receives it, creates a submission ID, and stores the original file.
- The system converts the file into one standard internal table format and flags any obvious data issues.
- The system suggests attribute values with confidence scores, then routes:
    - high confidence items to automatic approval
    - low confidence items to the human review queue
- Approved values are written back to PIMS, and every change is recorded with who, when, and what changed.